ASSINGMENT 2: Exploring our Spotify Data Base
ATHANASIOS GOURDOMICHALIS

1. Objective

In this assignment, we will explore Spotify data by writing and executing SQL queries.

2. Prerequisites

We must use the pre-constracted database provided at the following link: *Kaggle Spotify SQLite Database*

3. Project Scope

We are required to construct a total of **12 SQL queries**. Cumulatively, these queries must satisfy the following technical criteria (**NOTE:** a single query can cover multiple criteria simultaneously):

- **Data Filtering**: At least 4 queries must include a **WHERE** clause, utilizing either numerical comparison or the **LIKE** operator.
- **Inner Joins**: At least 6 queries must perform an inner join **(JOIN / INNER JOIN)** between tables.
- **Outer Joins**: At least 2 queries must perform an outer join (note that SQLite only supports **LEFT OUTER JOIN**).
- **Set Operators**: At least 1 query must contain set operators **(UNION, UNION ALL, INTERSECT, or EXCEPT)**.
- **Grouping & Aggregation**: At least 4 queries must contain **GROUP BY** and **HAVING** clauses, utilizing aggregate functions **(MIN, MAX, AVG, COUNT)**.
- **Nested Queries**: At least 2 queries must incorporate nested queries (Subqueries).
- **Sorting**: At least 2 queries must feature an **ORDER BY** clause. This can be combined with the **LIMIT** *k* clause to restrict the result set to the top *k* records.
- **Duplicate Elimination**: At least 1 query must utilize the **DISTINCT** keyword.

4. Implementation Criteria Checklist

Metric / Feature	**Minimum Required Count #**
Use of **WHERE** with numerical comparison or LIKE	**4**
Table joins using **INNER JOIN**	**6**
Use of LEFT OUTER JOIN (or **LEFT JOIN**)	**2**
Use of **UNION, UNION ALL, INTERSECT,** or **EXCEPT**	**1**
Use of **GROUP BY & HAVING** with **MIN, MAX, AVG, COUNT**	**4**
Use of Subqueries (**nested queries**)	**2**
Use of ORDER BY clause (**ASC/DESC**)	**2**
Use of **DISTINCT** keyword	**1**

5. Tools

If you prefer inspecting your tables in a graphical user interface rather than the command-line interface, you may use **DB Browser for SQLite**: sqlitebrowser.org

6. Instructions & Guidelines

- Store all SQL queries into a single file named ***queries.sql***.
  Include a brief description of each query and the exact number of returned records inside the file as comments. For example:

```sql
-- Retrieve the names of albums associated with the artist "Berliner Philharmoniker" (20)
SELECT DISTINCT al.name AS album_name
FROM     artists AS a
JOIN       r_albums_artists AS aa ON a.id = aa.artist_id
JOIN       albums AS al ON aa.album_id = al.id
WHERE  a.name = 'Berliner Philharmoniker';
```

7. Tips and Notes

- Execute and verify every query in your database.
- The descriptive comment must immediately precede its respective query. Use the -- notation for comment lines and terminate every SQL statement with a semicolon (;).
- Ensure that the queries retrieve meaningful and non-trivial information rather than simply projecting an entire table.
- Query results must be human-readable and comprehensible. For example, a query returning *"the IDs of the 10 most popular artists"* is not acceptable. Instead, the query must return *"the names of the 10 most popular artists"*.

8. Usefool Links:

- **SQLite SELECT Statement Syntax:** https://www.sqlite.org/lang_select.html

9. Project Files

- ***queries.sql***: Containing all the DML (Data Modifying Language) SQL queries.